

Assignment 1

Dilipkumar A

1 Process Abstraction and the POSIX API

1.1 State Machines Under Pressure

1.1(a)

When the hardware timer fires while a user process is running, the kernel performs the following sequence.

1. The timer interrupt causes the CPU to enter kernel mode through the interrupt/trap vector. The hardware saves the interrupted user context, including the program counter, processor status, stack pointer, and relevant registers, into a trap frame on the current process's kernel stack.
2. The timer interrupt handler runs. It acknowledges the timer device, updates kernel time/accounting data, and charges the current process for the elapsed CPU tick.
3. The kernel consults the current process's PCB/process descriptor and scheduler metadata. Since the process was in a tight arithmetic loop, it is not blocked; it is still runnable.
4. If the time slice has expired or the scheduler otherwise decides to preempt, the kernel records the process's saved CPU context in its PCB/context field. This includes the register set, program counter, stack pointer, and scheduling/accounting fields.
5. The kernel changes the current process's state in its PCB from *Running* to *Ready* and inserts that PCB into the appropriate run queue.
6. The scheduler reads the run queue and scheduling data structures, selects the next runnable process, removes that process's PCB from the run queue, and changes its state from *Ready* to *Running*.
7. The kernel updates the per-CPU current-process pointer to the selected PCB. If the next process is in a different address space, the kernel also switches address-space metadata, such as the page-table base register, and flushes or updates TLB state as required.
8. The low-level context-switch code switches to the next process's saved kernel stack and restores its saved context from its PCB.
9. The kernel returns from the interrupt/trap. The CPU resumes in user mode at the restored program counter of the newly selected process.

The key data structures read or written are the interrupt/trap frame, the current process's kernel stack, the PCB/process descriptor, the saved context field inside the PCB, scheduler accounting fields, the run queue, the per-CPU current pointer, and possibly the address-space/page-table data structures.

1.1(b)

The proposed sequence is

$$Running \rightarrow Terminated \rightarrow Blocked \rightarrow Ready \rightarrow Running.$$

i. The invalid transitions are:

- *Running* $\rightarrow$ *Terminated* is invalid for this scenario. A process that calls `read()` on a socket with no data available does not terminate; it voluntarily enters the kernel and may block waiting for the I/O event.
- *Terminated* $\rightarrow$ *Blocked* is impossible in the kernel state machine. A terminated process no longer executes and cannot issue a blocking request. Once a process terminates, it can only remain as a zombie until reaped; it cannot become blocked or runnable again.

The transitions *Blocked* $\rightarrow$ *Ready* and *Ready* $\rightarrow$ *Running* are valid in general, but they cannot validly follow from the erroneous terminated state in the proposed sequence.

ii. The correct sequence is

$$Running \rightarrow Blocked \rightarrow Ready \rightarrow Running.$$

More precisely:

- *Running* $\rightarrow$ *Blocked*: the process invokes `read()`. The system call originates from the process itself, and the OS changes the process state to blocked because the socket has no available data.
- *Blocked* $\rightarrow$ *Ready*: network data arrives. The hardware raises a device interrupt; the network interrupt handler and kernel networking code wake the sleeping process and move its PCB to the run queue.
- *Ready* $\rightarrow$ *Running*: the OS scheduler dispatches the process when it is selected from the run queue.

1.1(c)

A zombie process still occupies a process-table entry because the kernel must preserve the information the parent is entitled to collect: the child's PID, exit status, termination reason, resource-usage information, and parent/child linkage. Although the zombie is not executing and owns no CPU context that can be scheduled, its PCB entry cannot be fully destroyed until the parent collects that status.

The zombie is removed by a successful `wait()`, `waitpid()`, or related wait-family system call. This is called reaping the child. If the parent exits before collecting the zombie, the child is reaped to `init/systemd` or to the nearest configured subreaper. That new parent is responsible for waiting on and reaping the zombie.

1.2 Deep fork() Analysis

1.2(a)

If `fork()` is executed inside a loop that runs n times and there is no `exit()` inside the loop, then every existing process executes the next iteration. Therefore the number of processes doubles on each iteration:

$$N(n) = 2^n.$$

Equivalently, the number of newly created processes is $2^n - 1$, because the original process already existed before the loop.

The program in the assignment does not follow that unconstrained loop pattern. It has two syntactic `fork()` calls, but the second fork is inside the `if (rc1 == 0)` branch, so only the child of Fork A executes Fork B. Thus:

- Fork A creates one child, giving processes A and B.
- Only B executes Fork B, creating C.

So the actual program creates two new processes and has three processes total: A, B, and C. It is not the same as two unconstrained loop iterations, which would give $2^2 = 4$ processes.

1.2(b)

The exact output values are

C: `g=110 x=105`

B: `g=10 x=5`

A: `g=10 x=5`

Process C prints the first line. It is the child created by Fork B, so it executes

$$g \leftarrow g + 100, \quad x \leftarrow x + 100.$$

Its private copies therefore become $g = 110$ and $x = 105$.

Process B prints the second line after waiting for C. B did not execute C's increments. After `fork()`, each process has a separate virtual address space. The global variable `g` and local variable `x` are copied into the child address space, normally using copy-on-write pages. C's later writes affect only C's copy, not B's copy. Thus B still has $g = 10$ and $x = 5$.

Process A prints the third line after waiting for B. A did not execute either child's increments, so A also still has $g = 10$ and $x = 5$.

1.2(c)

The only possible ordering is

$$C \prec B \prec A.$$

The reason is the happens-before chain:

C prints $\rightarrow$ C exits $\rightarrow$ B's wait returns $\rightarrow$ B prints $\rightarrow$ B exits $\rightarrow$ A's wait returns $\rightarrow$ A prints.

Therefore every other permutation is impossible:

- B, C, A is impossible because B cannot print before C exits, and C prints before it exits.
- B, A, C is impossible because B cannot print before C, and A cannot print before B exits.
- A, B, C is impossible because A waits for B to exit.
- A, C, B is impossible because A waits for B to exit, and B waits for C to exit before printing.
- C, A, B is impossible because A waits for B to exit, so A cannot print before B.

1.2(d)

- i. No. Replacing `x` by a heap pointer does not make the child and grandchild share one ordinary mutable heap object. After `fork()`, the pointer value is copied, so the parent and child may contain the same virtual address value. However, they are in separate virtual address spaces. The heap pages are normally shared only as copy-on-write physical pages until one process writes to them; after a write, the OS gives the writing process its own private copy of the page. Thus this is not shared heap memory in the semantic sense.
- ii. In this fork-only program, a parent calling `free(x)` does not invalidate the child's private copy of the heap object, so the parent's `free()` does not by itself create a cross-process use-after-free in the child. The class of bug that would arise in a genuinely shared setting, such as threads or `mmap()` with `MAP_SHARED`, is a dangling-pointer/use-after-free bug. It can be detected with tools such as AddressSanitizer, Valgrind/Memcheck, heap debugging allocators, and stress tests that vary scheduling timing.
- iii. If B exits while C is still running and B has not waited for C, then C becomes an orphan, not a zombie. Immediately after B exits, C remains in whatever executable state it was already in, such as *Running*, *Ready*, or *Blocked*; the key change is its parent pointer. The OS orphan-reparenting mechanism reparents C to `init/systemd` or to the nearest subreaper. When C later terminates, that new parent is responsible for reaping it. If this pattern happens repeatedly and children are not reaped promptly, the system risks accumulating zombies or long-lived orphaned processes, which can exhaust PID/process-table entries and other per-process resources.

2 Scheduling Algorithms and Trade-offs

2.1 Multi-Process Workload Analysis

The workload is:

Process	Arrival	Burst	Priority
P1	0	10	3
P2	2	4	1
P3	4	6	2
P4	6	2	4
P5	8	8	2

2.1(a)

- i. Under SRTF, the only required preemption occurs at $t = 2$. From $t = 0$ to $t = 2$, P1 runs and has 8 ms remaining. At $t = 2$, P2 arrives with burst 4 ms. Since $4 < 8$, SRTF must preempt P1 and run P2.

There are later scheduling decision points, but they are not preemptions. At $t = 4$, P3 arrives, but P2 has only 2 ms remaining, so P2 continues. At $t = 6$, P2 finishes and P4 arrives; the scheduler chooses P4, but no running process is preempted because P2 has completed. At $t = 8$, P4 finishes and P5 arrives; again, this is a completion/dispatch event, not a preemption.
- ii. For this specific workload, P1 can be the last to finish only if the SRTF tie at $t = 14$ is broken against P1. At that time, P1 and P5 both have 8 ms remaining. With arrival-order

tie-breaking, P1 runs before P5, so P5 finishes last. With a tie-breaking rule that chooses P5 over P1, P1 would finish last.

Structurally, P1 is guaranteed to finish last if, until all other jobs have completed, there is always some other ready job with strictly smaller remaining time than P1, or an equal remaining time with ties consistently broken against P1. P1 is impossible to finish last once it becomes the unique job with the strictly smallest remaining time and no future arrival can introduce a shorter remaining time; SRTF will then keep P1 running until it completes.

2.1(b)

FCFS must run P1 first because P1 is the only job at $t = 0$. After P1 completes, all other jobs have arrived. FCFS would keep arrival order among the remaining jobs, placing P2 and P3 before P4 even though P4 is the shortest remaining job. Non-preemptive SJF also runs P1 first, but then chooses the available jobs in increasing burst length, moving P4 ahead of P2 and P3. This removes long-before-short inversions from the FCFS order. Moving a shorter job before longer jobs reduces the shorter job's wait more than it increases the longer jobs' waits, so the average wait time under non-preemptive SJF is lower than or equal to FCFS for this workload. The process responsible for the difference is P4, because it is short but arrives after P2 and P3.

2.1(c)

Assumption for Round Robin: when a process arrives at exactly the same time a quantum expires, the newly arrived process is enqueued before the just-expired process is requeued. With $q = 3$ ms, the schedule is

Time	0–3	3–6	6–9	9–12	12–14	14–15
Process	P1	P2	P1	P3	P4	P2

Time	15–18	18–21	21–24	24–27	27–28	28–30
Process	P5	P1	P3	P5	P1	P5

Process	Arrival	Burst	Finish	Turnaround	Wait
P1	0	10	28	28	18
P2	2	4	15	13	9
P3	4	6	24	20	14
P4	6	2	14	8	6
P5	8	8	30	22	14

$$\text{Average wait} = \frac{18 + 9 + 14 + 6 + 14}{5} = \frac{61}{5} = 12.2 \text{ ms.}$$

$$\text{Average turnaround} = \frac{28 + 13 + 20 + 8 + 22}{5} = \frac{91}{5} = 18.2 \text{ ms.}$$

Round Robin has higher average turnaround than SRTF on this workload because it repeatedly time-slices the CPU instead of completing the shortest remaining jobs as soon as possible. That improves first-response fairness, but it delays completion of short jobs such as P2, P4, and P3. SRTF minimizes completion latency for short remaining jobs, while RR spreads service across all ready jobs.

2.1(d)

For non-preemptive SJF, P1 must run first because it is the only job available at $t = 0$. At $t = 10$, all other jobs have arrived, so SJF chooses them by increasing burst time:

P1 (0-10), P4 (10-12), P2 (12-16), P3 (16-22), P5 (22-30).

Process	Arrival	Burst	Start	Finish	Wait	Turnaround
P1	0	10	0	10	0	10
P2	2	4	12	16	10	14
P3	4	6	16	22	12	18
P4	6	2	10	12	4	6
P5	8	8	22	30	14	22

$$\text{Average wait} = \frac{0 + 10 + 12 + 4 + 14}{5} = 8 \text{ ms.}$$

$$\text{Average turnaround} = \frac{10 + 14 + 18 + 6 + 22}{5} = 14 \text{ ms.}$$

2.2 Theoretical Bounds

2.2(a)

Let the three burst times satisfy $b_1 \leq b_2 \leq b_3$, and suppose all jobs arrive at $T = 0$. For an order (i, j, k) , the waiting times are

$$0, \quad b_i, \quad b_i + b_j,$$

so the average wait is

$$\overline{W}(i, j, k) = \frac{2b_i + b_j}{3}.$$

SJF uses order $(1, 2, 3)$, so

$$\overline{W}_{SJF} = \frac{2b_1 + b_2}{3}.$$

Compare every other possible ordering:

$$\begin{aligned} \overline{W}(1, 3, 2) - \overline{W}_{SJF} &= \frac{b_3 - b_2}{3} \geq 0, \\ \overline{W}(2, 1, 3) - \overline{W}_{SJF} &= \frac{b_2 - b_1}{3} \geq 0, \\ \overline{W}(2, 3, 1) - \overline{W}_{SJF} &= \frac{b_2 + b_3 - 2b_1}{3} \geq 0, \\ \overline{W}(3, 1, 2) - \overline{W}_{SJF} &= \frac{2b_3 - b_1 - b_2}{3} \geq 0, \\ \overline{W}(3, 2, 1) - \overline{W}_{SJF} &= \frac{2(b_3 - b_1)}{3} \geq 0. \end{aligned}$$

Thus no non-preemptive ordering has smaller average wait than SJF. Equality is possible only when the compared burst times are equal.

2.2(b)

Use the standard OS definition of response time: time from arrival until the job first receives CPU service.

Under SRTF, the k short jobs run before the long job. Their first response times are

$$0, s, 2s, \dots, (k-1)s,$$

and the long job's first response time is ks . Therefore

$$\overline{R}_{SRTF} = \frac{s(0 + 1 + \dots + (k-1)) + ks}{k+1} = \frac{s \cdot \frac{k(k-1)}{2} + ks}{k+1} = \frac{ks}{2}.$$

Under Round Robin with quantum $q < s$, each of the $k+1$ jobs receives a first quantum during the first round. Regardless of the tie order among simultaneous arrivals, the first response times are

$$0, q, 2q, \dots, kq.$$

Hence

$$\overline{R}_{RR} = \frac{q(0 + 1 + \dots + k)}{k+1} = \frac{q \cdot \frac{k(k+1)}{2}}{k+1} = \frac{kq}{2}.$$

RR has lower average response time when

$$\frac{kq}{2} < \frac{ks}{2}.$$

For $k > 0$, this reduces to

$$q < s.$$

That is exactly the stated case, so RR gives lower average first-response time for any long-job size L satisfying the intended “long job” assumption $L > s$. The value of L does not appear in the first-response-time formula because response time ends when the job first receives CPU service, not when it completes. Intuitively, SRTF makes the long job wait until all short jobs finish, whereas RR gives every job some CPU within one round of quanta.

3 Quantitative Performance and Advanced Metrics

3.1 Metric Derivation

All processes arrive at $T = 0$, so wait time is the start time and turnaround time is the finish time.

3.1(a)

Process	Start	Finish	Wait	Turnaround
P1	0	12	0	12
P2	12	15	12	15
P3	15	22	15	22
P4	22	24	22	24
P5	24	30	24	30

3.1(b)

$$\text{Makespan} = \max_i f_i - \min_i a_i = 30 - 0 = 30 \text{ ms.}$$

3.1(c)

The wait times are

$$0, 12, 15, 22, 24.$$

Thus

$$\begin{aligned} \sum x_i &= 0 + 12 + 15 + 22 + 24 = 73, \\ \sum x_i^2 &= 0^2 + 12^2 + 15^2 + 22^2 + 24^2 = 0 + 144 + 225 + 484 + 576 = 1429. \end{aligned}$$

With $n = 5$,

$$J = \frac{(\sum x_i)^2}{n \sum x_i^2} = \frac{73^2}{5(1429)} = \frac{5329}{7145} \approx 0.746.$$

3.1(d)

The burst times in the FCFS trace are $P1 = 12$, $P2 = 3$, $P3 = 7$, $P4 = 2$, and $P5 = 6$. SJF would move the shorter jobs toward the front and push the longest job, $P1$, to the end. This lowers mean wait but makes the wait distribution less equal. Jain's index measures equality of waits, not average wait, so the SJF fairness index is lower than the FCFS value.

For confirmation, SJF order is $P4, P2, P5, P3, P1$, giving wait times

$$0, 2, 5, 11, 18.$$

Then

$$J_{SJF} = \frac{36^2}{5(0^2 + 2^2 + 5^2 + 11^2 + 18^2)} = \frac{1296}{2370} \approx 0.547 < 0.746 = J_{FCFS}.$$

Therefore $J_{SJF} < J_{FCFS}$.

3.2 Interpreting Real-World Measurements

3.2(a)

Scheduler Beta is most consistent with a starvation-prone shortest-job/shortest-remaining-time or strict-priority scheduling family. Such schedulers can keep the mean wait very low by serving many short or high-priority jobs quickly, while a small number of long or low-priority jobs suffer extreme waits.

A concrete example is one long job and many short jobs under SRTF:

- L : arrival 0, burst 1000.
- $S_1, \dots, S_{100}$: one short job of burst 1 arrives at each time $0, 1, 2, \dots, 99$.

SRTF keeps selecting the short jobs. Almost all short jobs wait about 0, while L waits until the short stream ends. The mean wait is low because 100 of the 101 jobs have tiny wait times, but the tail wait is extreme because the long job absorbs nearly all the waiting. This is the same qualitative pattern as Beta: low mean wait but very large $P90/P99$ wait.

3.2(b)

A scheduler can minimize makespan by maximizing throughput: keeping CPUs busy, reducing context-switch overhead, improving cache locality, batching similar work, or prioritizing jobs that make the whole batch finish sooner. Those choices can still make individual processes wait longer on average if the scheduler concentrates waiting on some jobs or delays interactive jobs to improve global completion time. Makespan is therefore useful for throughput-oriented batch workloads, but it is not sufficient for interactive workloads. Interactive systems care about response and tail latency, so P90 and P99 wait are more informative than makespan alone.

3.2(c)

For the overnight batch genomics pipeline, choose **Gamma**. Its makespan is 870 ms, the lowest of all four schedulers, so it has the best throughput. Its tail values, P90 = 14 ms and P99 = 19 ms, are also low, so the throughput gain does not come with a severe tail-latency penalty.

For the real-time chat application, choose **Delta**. The visible-delay threshold is 50 ms, so tail latency matters more than average throughput. Delta has the lowest P90 wait, 11 ms, and the lowest P99 wait, 16 ms, both far below 50 ms. Its mean wait is 9 ms, also low. Beta must be rejected despite its mean wait of 6 ms, because its P90 wait of 280 ms and P99 wait of 620 ms are far above the chat application's delay threshold.